

RAPPORT DE PROJET

Pipeline de Traitement ETL Distribue avec Apache Spark et Scala

Intégration, Transformation et Chargement de Données Multi-Sources

Auteur : Achraf Kaou
Enseignant: Nejla Essadi
Master en Ingenieurie des Logicielles Libres
Année Universitaire : 2025/2026

1. Introduction

1.1 Contexte

Dans un monde où les données sont générées à une vitesse exponentielle, les entreprises doivent traiter des volumes massifs provenant de sources hétérogènes (fichiers plats, bases de données, APIs). Les pipelines ETL (Extract, Transform, Load) traditionnels atteignent rapidement leurs limites en termes de scalabilité et de performance.

1.2 Problématique

Les principales difficultés rencontrées sont :

- Hétérogénéité des sources de données (formats, schémas, qualité variable)
- Volumes importants nécessitant un traitement distribué
- Besoin de nettoyage, déduplication et enrichissement des données
- Exigence de chargement incrémental et cohérent dans un Data Warehouse
- Gestion robuste des erreurs et traçabilité

1.3 Objectifs du projet

Ce projet vise à concevoir et implémenter un pipeline ETL complet, scalable et robuste en utilisant Apache Spark. Les objectifs incluent la maîtrise des lectures/écritures multi-formats, les transformations complexes, la modélisation en schéma en étoile et la gestion de la qualité des données.

1.4 Technologies utilisées

- **Apache Spark** (Scala) : Moteur de traitement distribué
- **Docker** : Conteneurisation des environnements et bases de données

- **PostgreSQL** : Base source et cible
- **SBT** : Outil de build Scala

Ce rapport détaille l'architecture, l'implémentation et les résultats obtenus.

2. Analyse des Besoins du Projet

Le projet demande l'intégration de données provenant de :

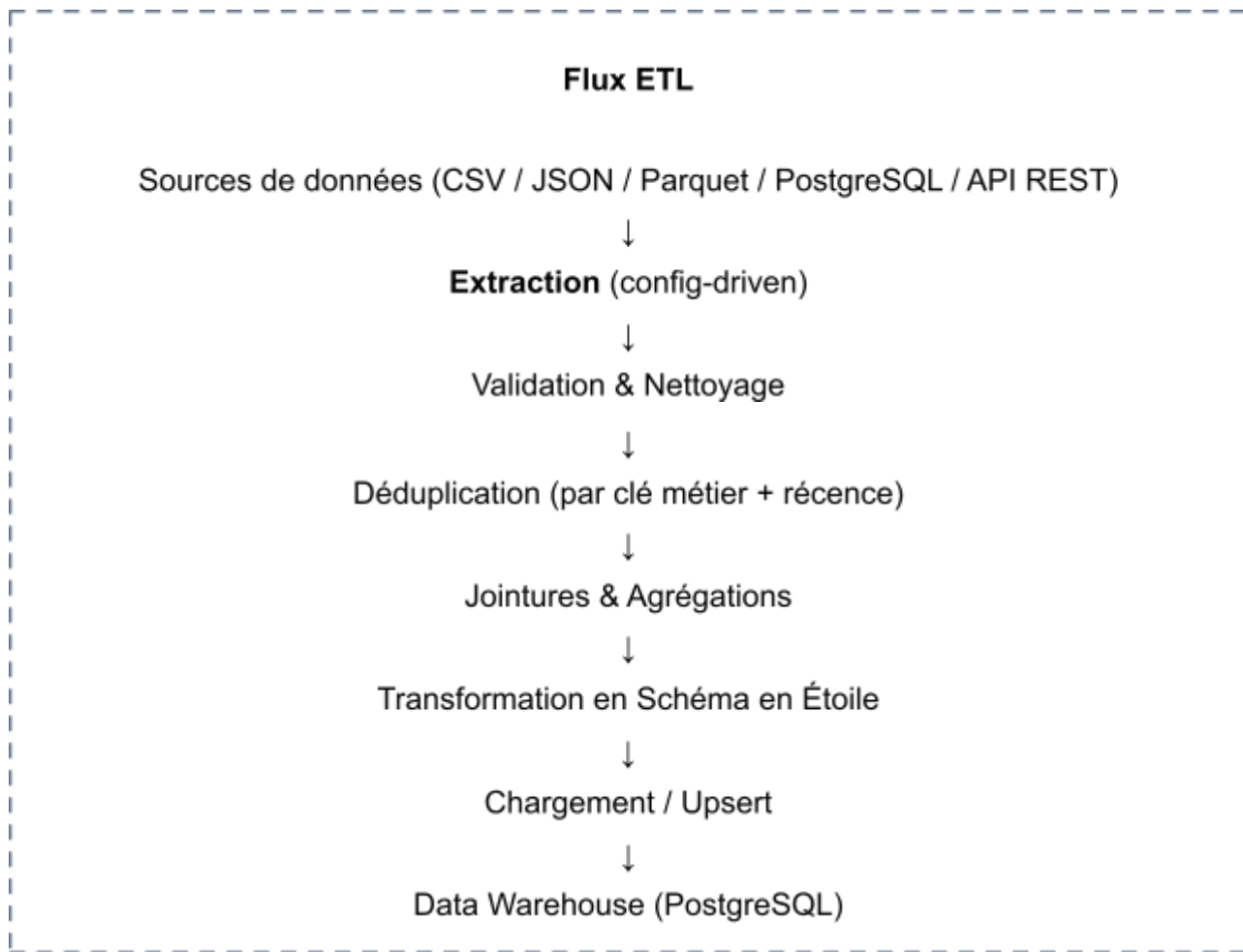
- Fichiers plats (CSV, JSON, Parquet)
- Base de données relationnelle (via JDBC)
- API REST

Les fonctionnalités attendues sont :

Exigence	Solution Implémentée
Extraction fichiers plats	Lecteurs Spark SQL avec inférence de schéma
Intégration API	Extracteurs REST avec parsing JSON
Connexion BDD	Connecteurs JDBC
Nettoyage & Déduplication	Transformations configurables + priorité des sources
Chargement Warehouse	Système d'upsert sur clés métier
Orchestration	Pipeline modulaire avec dépendances
Journalisation	Logs structurés + métriques

Le modèle cible suit un schéma en étoile pour faciliter l'analyse décisionnelle.

3. Architecture Générale du Pipeline



L'architecture est modulaire : chaque étape (extractor, transformer, loader) est encapsulée dans des classes Scala séparées. Le pipeline est piloté par un fichier de configuration (application.conf ou JSON) permettant d'ajouter/supprimer des sources sans modifier le code.

Avantages : Séparation des responsabilités, réutilisabilité, testabilité et scalabilité horizontale grâce à Spark.

4. Technologies et Outils Utilisés

Technologie	Raison du choix
Apache Spark (Scala)	Traitement distribué, API DataFrame/Spark SQL performante, typage fort
Docker Compose	Environnement reproductible (Spark + PostgreSQL + services mock)

PostgreSQL	Simulation de sources et entrepôt de données cible
SBT	Gestion des dépendances et packaging du projet Scala

Spark a été choisi pour sa capacité à traiter des téraoctets de données sur un cluster tout en offrant une API unifiée pour le batch ETL.

5. Phase d'Extraction

Les extracteurs sont configurables via un objet `SourceConfig`.

5.1 Fichiers plats

```
spark.read
  .option("header", "true")
  .option("inferSchema", "true")
  .csv("/data/sales_*.csv")
```

Support de CSV, JSON et Parquet avec détection automatique du schéma.

5.2 API REST

Utilisation de `requests` Scala ou Spark avec `spark.read.json` après appel HTTP et parsing.

5.3 Bases de données

```
spark.read
  .format("jdbc")
  .option("url", jdbcUrl)
  .option("dbtable", table)
  .option("user", user)
  .load()
```

Toutes les extractions sont tracées avec un timestamp et un identifiant de run.

6. Phase de Transformation

6.1 Nettoyage des données

- Gestion des valeurs nulles (imputation ou suppression)
- Normalisation (trim, lower case, formatage dates)

- Validation de règles métier (filtre sur colonnes invalides)

6.2 Déduplication

Stratégie : priorité source + récence (colonne `ingestion_date`).

```
df.withColumn("rank", rank().over(Window.partitionBy("id_client").orderBy(col("ingestion_date").desc()))  
  .filter("rank <= 1")  
  .drop("rank")
```

6.3 Jointures et Agrégations

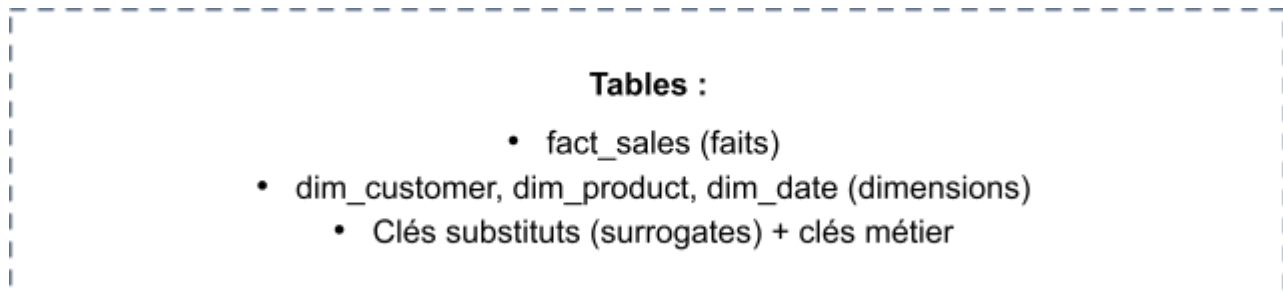
Jointures sur clés composites avec gestion des conflits de colonnes via alias. Agrégations pour calculer des métriques (CA, quantité moyenne, etc.).

Exemple de transformation métier :

```
val enriched = sales.join(customers, "client_id", "left")  
  .withColumn("montant_ttc", col("montant_ht") * (1 + col("tva")/100))
```

7. Data Warehouse et Chargement

Le modèle cible est un **schéma en étoile** :



Upsert (Merge)

Implémenté via :

- `MERGE INTO` (Delta Lake recommandé en production)
- Ou logique SQL classique : `INSERT ... ON CONFLICT` (PostgreSQL)

Le chargement est incrémental et idempotent.

8. Orchestration, Logs et Gestion des Erreurs

Orchestration via une classe `ETLPipeline` qui enchaîne les étapes avec gestion des dépendances.

Journalisation : Utilisation de Log4j/SLF4J avec logs structurés (JSON) + métriques Spark (nombre de lignes, temps d'exécution par étape).

Gestion des erreurs :

- Try/Catch par étape avec fallback
 - Fail-fast pour erreurs critiques
 - Stockage des enregistrements rejetés dans une table `error_log`
-

9. Tests et Résultats

stratégie de test :

- Tests unitaires (ScalaTest) sur chaque transformateur
- Tests d'intégration avec données mockées
- Validation de bout en bout via Docker Compose

Performances observées :

- Traitement de 1 million de lignes en quelques minutes sur un cluster local
- Partitionnement optimal réduit les shuffles
- Scalabilité linéaire avec ajout de workers Spark

Captures d'écran (exécution Spark UI, tables finales dans pgAdmin) sont disponibles dans le dépôt.

10. Conclusion et Perspectives

Ce projet a permis de concevoir un pipeline ETL distribué complet, modulaire et robuste respectant les bonnes pratiques du domaine.

Objectifs atteints : Maîtrise de Spark, gestion multi-sources, schéma en étoile, upserts, orchestration.

Perspectives d'amélioration :

- Intégration Apache Airflow pour l'orchestration avancée
- Passage en streaming avec Spark Structured Streaming + Kafka
- Utilisation de Delta Lake pour ACID et Time Travel

- Déploiement sur Kubernetes / Databricks / AWS EMR
- Monitoring avec Prometheus + Grafana

Le code source, les scripts Docker et la documentation complète sont disponibles sur le dépôt GitHub du projet.